

Natural Language Processing: Dialog Systems and Chatbots

Alessandro ARENSBERG

May 22, 2026

Contents

1	Introduction	2
2	Datasets and Preprocessing	2
3	Exploratory Data Analysis	2
4	Training Procedure	2
5	Sample Conversations	2
6	Exercise 1: Data Preprocessing, Exploration, and Baseline Chatbot	3
7	Exercise 2: Improved Neural Chatbot and Decoding Strategy	3
8	Exercise 3: Training on Multiple Datasets and Domain Effects	4
9	Exercises 4 to 6: Memory, Personality, and Feedback	4
9.1	Exercise 4: Conversational Memory and Context Handling	4
9.2	Exercise 5: Chatbot Personality and Consistency	4
9.3	Exercise 6: User Feedback Loop and Adaptation	5
10	Quantitative Evaluation	5
11	Qualitative Evaluation	5
12	Exercise 7: Evaluation, Ablation Study, and Error Analysis	6
13	Ablation Study	6
14	Error Analysis	7
15	Limitations and Ethics	7
16	Possible Future Improvements	7
17	Conclusion	7

1 Introduction

This coursework develops a reproducible chatbot system in PyTorch from a small Cornell Movie-Dialogs baseline to a more complete evaluation stack with attention, multi-dataset training, conversational memory, persona control, user feedback, and final ablation analysis. The focus is not only on model quality, but also on transparent experimental design, reproducibility, and critical interpretation of the outputs.

2 Datasets and Preprocessing

The main training corpus is Cornell Movie-Dialogs. Conversation chains are converted into single-turn input-response pairs by linking each utterance to the next one in the same conversation. The preprocessing pipeline lowercases text, tokenizes punctuation, filters very short or excessively long examples, and splits by conversation rather than by pair to reduce leakage across train, validation, and test sets.

The same preprocessing style is reused for the later experiments when PersonaChat is added. The project also keeps DailyDialog, EmpatheticDialogues, and OpenSubtitles available in the workspace, but the final coursework experiments mainly rely on Cornell and PersonaChat.

3 Exploratory Data Analysis

The exploratory analysis is centered on the Cornell baseline corpus and records the scale and shape of the dialogue data before modelling. The final dataset summary reports 66,962 conversations, 149,051 pairs, a vocabulary size of 36,440, and an average source/target length of 8.41 tokens. The saved plots in the artifact folders show the expected dialogue skew toward short conversational turns and common function words.

The sample outputs also show typical noise patterns in movie dialogue, such as punctuation-heavy fragments, short acknowledgements, and weakly informative replies. This is useful for the report because it explains why the baseline chatbot struggles with generic responses.

4 Training Procedure

Training is carried out in small, reproducible stages. Exercise 1 uses the baseline Cornell pipeline with a fixed seed and a modest GRU encoder-decoder setup. Exercise 2 trains the attention-based model and compares the GloVe and no-GloVe variants. Exercise 3 reuses the same attention model but changes the data mixture by adding PersonaChat. Exercises 4 to 6 do not retrain the neural generator; instead, they wrap the previously trained checkpoint with memory, persona, and feedback logic.

The main runs are documented in the README with exact commands, batch sizes, device settings, and output directories. This keeps the report aligned with the executable workflow and makes the results reproducible.

5 Sample Conversations

Sample conversations are included for the baseline, improved, memory, persona, and feedback experiments through the artifact files under `artifacts/exercise1/outputs`, `artifacts/exercise2/outputs`, `artifacts/exercise3/outputs`, `artifacts/exercise5/outputs`, and `artifacts/exercise6/outputs`. They illustrate the key qualitative differences between the variants: the baseline is weak and

generic, the improved model is more fluent, the memory layer can answer factual recall questions, the persona layer preserves a stable identity, and the feedback layer can override a bad response with a stored correction.

6 Exercise 1: Data Preprocessing, Exploration, and Baseline Chatbot

Exercise 1 establishes the baseline pipeline and the first PyTorch chatbot. The model is a word-level GRU encoder-decoder trained with teacher forcing and decoded with greedy or beam search. The purpose of this stage is to obtain a clean reference point before adding more sophisticated mechanisms.

The preprocessing output is fully deterministic with a fixed random seed. The pipeline writes the split manifest, training history, vocabulary summary, sample conversations, and EDA figures under `artifacts/exercise1/outputs`. The baseline checkpoint is stored in `artifacts/exercise1/checkpoints/baseline_seq2seq.pt`.

Final Cornell baseline metrics:

- Test loss: 4.1988
- Test perplexity: 66.6057
- Vocabulary size: 36,440
- Average source and target length: 8.41 tokens

The qualitative samples are intentionally weak, which is useful for the report because it exposes the limitations of a plain seq2seq model on movie dialogue.

7 Exercise 2: Improved Neural Chatbot and Decoding Strategy

Exercise 2 improves the baseline with a multi-layer bidirectional GRU encoder, additive attention in the decoder, optional GloVe initialisation, and a richer decoding interface. Three decoding strategies are compared: greedy decoding, beam search, and top-k sampling with temperature.

The best validated reference run uses the extracted WikiGiga GloVe vectors. The improvement over the baseline is visible both in loss and in generation quality, although the model remains relatively weak in absolute terms.

Key results for the full Cornell run with GloVe:

- Train loss: 4.3799
- Validation loss: 4.0673
- Test loss: 4.0786
- Test perplexity: 59.0631
- Greedy BLEU: 0.0059, distinct-1: 0.0028, distinct-2: 0.0110
- Beam BLEU: 0.0054, distinct-1: 0.0040, distinct-2: 0.0137
- Top-k + temperature BLEU: 0.0052, distinct-1: 0.0110, distinct-2: 0.1079

A matched no-GloVe run is kept as the ablation reference. It reaches test loss 4.1441 and test perplexity 63.0630, which confirms that GloVe improves validation and test loss, even though the no-GloVe run is occasionally slightly stronger on greedy and beam BLEU.

8 Exercise 3: Training on Multiple Datasets and Domain Effects

Exercise 3 compares Cornell-only training with Cornell plus PersonaChat using the same attention-based seq2seq architecture. The source text is prefixed with a corpus tag so the model can condition on the domain without changing the core network.

The final mixed-domain run was completed and is stored in `artifacts/exercise3/outputs`. The comparison shows that adding PersonaChat improves the language model loss and keeps the pipeline reproducible across corpora.

Final mixed-domain metrics:

- Train loss: 4.3592
- Validation loss: 4.0182
- Test loss: 4.0382
- Test perplexity: 56.7258
- Cornell domain loss: 4.0711
- PersonaChat domain loss: 3.9641

On generation quality, the mixed-domain model improves over the Cornell-only control, and top-k sampling remains the main driver of output diversity. The artifact `artifacts/exercise3/outputs/comparison` contains the full comparison.

9 Exercises 4 to 6: Memory, Personality, and Feedback

These three exercises deliberately move away from pure neural generation and add deterministic control layers on top of the trained chatbot. That design choice keeps each component isolated and makes the behaviour easy to verify.

9.1 Exercise 4: Conversational Memory and Context Handling

Exercise 4 implements a rule-based conversational memory in `src/memory_ex4.py`. The memory state tracks names, locations, preferences, and recent statements, and answers direct recall questions without relying on the seq2seq model. A small vector-based extension is also available for paraphrased recall cases.

This is intentionally simple and transparent: it is deterministic, inspectable, and does not require retraining the chatbot. The memory-aware wrapper only falls back to the neural model when no direct memory answer is available.

The smoke test covers five targeted cases and passes all of them. The validated artifacts are stored in `artifacts/exercise4/outputs`. The vector generalisation extension also passes its targeted paraphrase test set.

9.2 Exercise 5: Chatbot Personality and Consistency

Exercise 5 adds a stable persona profile named Ari. Consistency is enforced through three mechanisms: a fixed persona description, a persona-prefixed prompt, and rule-based overrides for direct identity questions and adversarial attempts to change the persona.

The six-case smoke test passes all cases, including repeated identity questions and adversarial prompts. This confirms that personality consistency is produced by explicit control, not by the raw generator alone. The outputs are stored in `artifacts/exercise5/outputs`.

9.3 Exercise 6: User Feedback Loop and Adaptation

Exercise 6 introduces a persistent `FeedbackStore` that maps normalized user queries to corrected responses. Corrections are written to `artifacts/exercise6/outputs/feedback_store.json` and are automatically consulted during inference. If a stored correction matches the query, the corrected response replaces the generator output.

The inference path integrates the feedback store into the CLI and HTTP server, so the system can be used interactively. The server exposes `/infer`, `/add_correction`, and `/feedback` endpoints.

The lookup strategy is lightweight and transparent. It combines character similarity and token overlap rather than using a neural retriever, which keeps the behaviour reproducible and easy to inspect.

Transparency note. Some Exercise 6 evaluation scripts use controlled fixtures to test the feedback mechanism directly. In particular, the canonical feedback corrections are seeded before paraphrase and generalization checks. This is intentional and documented: the goal is to validate the mechanism, not to hide model outputs or claim that the chatbot learned those responses during training.

The larger Exercise 6 protocol evaluates 23 cases, including 17 positive paraphrases and 6 negative controls. The current run reports:

- Positive paraphrase coverage: 0.8824
- False-positive rate on negative controls: 0.1667
- Overall accuracy: 0.8696

The main failure mode is semantic over-matching on self-description prompts and one unrelated control query that triggers a false positive. The full reports and before/after examples are stored under `artifacts/exercise6/outputs`.

10 Quantitative Evaluation

The quantitative picture is consistent across the experiments. Exercise 1 reaches test loss 4.1988 and test perplexity 66.6057, which serves as the baseline reference. Exercise 2 improves the full Cornell run to test loss 4.0786 and test perplexity 59.0631 with GloVe. Exercise 3 further improves the mixed-domain model to test loss 4.0382 and test perplexity 56.7258. The memory and persona tests both reach 5/5 and 6/6 accuracy respectively, while the feedback loop reaches 0.8824 positive paraphrase coverage and 0.8696 overall accuracy on the larger protocol.

These numbers show the intended trend: the stronger architecture and extra data help the neural model, while the wrapper components achieve perfect or near-perfect scores on their targeted test sets.

11 Qualitative Evaluation

Qualitatively, the baseline chatbot remains generic and weak, often producing short or repetitive replies. The improved model is still limited, but it is more coherent and less degenerate. The memory and persona layers produce the clearest qualitative improvements because they answer targeted questions directly and consistently. The feedback layer is also effective, but it exposes the main weakness of the system: semantic over-matching can produce false positives on unrelated prompts.

The qualitative examples in the artifact folders therefore play an important role in the report. They make the strengths and failure cases visible in a way that the automatic metrics alone cannot capture.

12 Exercise 7: Evaluation, Ablation Study, and Error Analysis

Exercise 7 consolidates the full system evaluation into one critical comparison. The script `src/evaluate_ex7.py` aggregates the results from the earlier exercises and writes `ex7_summary.json`, `ex7_report.txt`, and `ex7_examples.txt`.

The purpose of this stage is not to train a new model. It is to compare the baseline, improved, memory, persona, and feedback variants, and to identify the main strengths and weaknesses of the overall system.

Main comparative results:

- Exercise 1 baseline test loss: 4.1988, test perplexity: 66.6057
- Exercise 3 improved model test loss: 4.0382, test perplexity: 56.7258
- Exercise 4 memory accuracy: 5/5
- Exercise 5 persona accuracy: 6/6
- Exercise 6 feedback positive coverage: 0.8824
- Exercise 6 feedback false-positive rate: 0.1667
- Exercise 6 feedback overall accuracy: 0.8696

The ablation study shows the contribution of each component. Removing PersonaChat weakens the improved model, removing the rule-based memory drops factual recall, removing the persona rules breaks persona consistency, and the feedback loop improves targeted coverage at the cost of a small false-positive risk on unrelated prompts.

The main failure cases are concentrated in feedback generalization. Two self-description prompts are missed because the lightweight similarity matcher over-generalises toward the identity correction or fails to resolve the paraphrase. One unrelated control prompt is incorrectly matched, which is the main safety caveat for the feedback mechanism.

13 Ablation Study

The repository supports several transparent ablations rather than hidden architectural changes. The most important are:

- Cornell-only versus Cornell plus PersonaChat in Exercise 3.
- GloVe versus no-GloVe in Exercise 2.
- Baseline versus memory-aware inference in Exercise 4.
- Baseline versus persona-conditioned inference in Exercise 5.
- Before versus after feedback in Exercise 6.

These ablations make the effect of each design choice explicit and help separate genuine improvements from wrapper-based control logic.

14 Error Analysis

The system still shows typical chatbot failure modes: generic responses, punctuation-heavy outputs, weak semantic grounding, and a tendency to overfit to short dialogue patterns. These problems are expected for a word-level seq2seq model trained on a limited budget.

The feedback mechanism adds a different failure mode: semantic over-matching. This is acceptable for a lightweight coursework implementation, but it should be described honestly as a tradeoff rather than presented as robust factual reasoning.

15 Limitations and Ethics

The project is intentionally modest in scope. It does not use pretrained generative chatbots, and it does not claim to solve long-range conversation or factual grounding. The memory, persona, and feedback components are deterministic wrappers, which is a strength for reproducibility but also a limitation for generalization.

The report therefore distinguishes clearly between neural improvements, deterministic control logic, and controlled test fixtures. This avoids overstating the results and makes the evaluation defensible.

16 Possible Future Improvements

Several follow-up directions would make the system stronger while staying within the same coursework constraints.

- Train the neural chatbot for longer or with a larger batch schedule to improve fluency and reduce generic outputs.
- Explore a stronger sequence-to-sequence architecture, such as subword modelling or a compact Transformer trained from scratch.
- Add a more robust memory retrieval mechanism so paraphrased factual questions generalise beyond the handcrafted test cases.
- Improve the feedback store with conflict resolution, provenance tracking, and a stricter safety check before applying corrections.
- Extend the evaluation with human ratings, more diverse negative controls, and additional qualitative examples for failure analysis.
- Report decoding ablations more systematically, especially the tradeoff between greedy, beam, and stochastic sampling.

These extensions would mainly improve robustness, coverage, and evaluation depth, while keeping the project transparent and reproducible.

17 Conclusion

The final repository contains a complete coursework chatbot pipeline in PyTorch. Exercise 1 provides the baseline preprocessing and seq2seq model, Exercise 2 improves the architecture and decoding, Exercise 3 adds multi-dataset training, Exercise 4 introduces conversational memory, Exercise 5 enforces persona consistency, Exercise 6 adds a persistent feedback loop, and Exercise 7 consolidates the evaluation and error analysis.

The most important outcome is not a state-of-the-art chatbot, but a transparent and reproducible experimental progression. The system now has a clear baseline, a stronger neural model, explicit control layers, documented limitations, and a concrete future work path, which is exactly what the coursework asks for.